



Software Engineering Department

Braude College of Engineering

Capstone Project – Phase B (61999)

DeMaestro:

[Autonomous Multi-Agent System for Full-Stack Web Application Synthesis](#)

CODE: 26-1-D-25

By:

Owise Zoubi: owisezoubi@gmail.com

Mohamad Atamneh: Mohamad.atamneh@e.braude.ac.il

Advisor:

Dr. Natali Levi

Link to GitHub

<https://github.com/owisezoubi/DeMaestro>

Project URL:

<https://de-maestro-code-53sp.vercel.app/>

Project Code:

<https://github.com/owisezoubi/DeMaestro-Code>

Abstract

Building a full-stack web application needs clear requirements and careful work between the frontend, the backend, and the database. In real life, users describe their ideas in text or documents that are often not complete or not clear enough. This makes it hard to turn an idea into a working application. Existing AI tools help developers, but most of them focus on writing code or building things visually. They assume the requirements are already clear, they do not really check what the user wanted, and they do not show how the user's input became the final system. This project presents DeMaestro. It is a system that takes a user's requirements (either as plain text or as a PDF document) and produces a full-stack web application that can actually run. The system first reads the input and asks the user to clarify anything that is missing. It shows the user a summary and only starts writing code after the user approves it. The generated project is tested inside a subprocess sandbox on the server, and then it is pushed to Vercel with a fresh Neon Postgres database for that project. The user gets a live URL to open the app in the browser, and also a downloadable ZIP with the full source code that they can extend later.

1.0 Introduction.....	2
2.0 Background and Related Work.....	4
2.1 AI-Based Code Assistants.....	4
2.2 Natural Language–Driven Application Generation.....	4
2.3 No-Code and Low-Code Development Platforms.....	5
2.4 Multi-Agent LLM Systems.....	5
3.0 General Description.....	5
3.1 System Objectives.....	5
3.2 Target User Identification.....	6
3.3 The Agent Architecture: Centralized Orchestrator Multi-Agent Pattern.....	6
4.0 Development Process.....	8
4.1 Phase 1: Research and Planning.....	8
4.2 Phase 2: Architecture Design.....	8
4.3 Phase 3: The Development Phase.....	9
4.4 Phase 4: Integration and Testing.....	9
4.5 Phase 5: Refinement and Optimization.....	10
5.0 System Architecture.....	10
5.1 System Package Diagram.....	11
5.1.1 The Frontend Layer (Web Client).....	11
5.1.2 The Backend Orchestration Layer.....	12
5.1.3 The Data and Persistence Layer.....	12
5.1.4 The AI Agent Engine Layer.....	12
5.1.5 The Subprocess Execution and Cloud Deployment Layer.....	13
5.2 System Flow.....	14
5.3 Implementation Overview.....	15
6.0 Challenges and Obstacles.....	15
6.1 Enforcing Output Determinism from Generative AI.....	15
6.2 Host-Safe Execution via Subprocess Virtualization.....	16
6.3 The Debugger Strategy Ladder.....	16

6.4 Serverless Cloud Deployment via Vercel.....	17
6.5 Lessons Learned.....	17
7.0 Project Metrics.....	18
7.1 System Performance and Generation Time.....	18
7.2 Code Reliability and Validity Rate.....	18
7.3 Usability and Interface Effectiveness.....	19
References.....	20
Appendices.....	21
Appendix A – User Guide.....	21
A.1 Introduction.....	21
A.2 Operating Instructions.....	21
A.3 Navigation and Workspace Management.....	26
Appendix B – Maintenance Guide.....	26
B.1 Introduction.....	26
B.2 System Requirements.....	26
B.3 Configuration and API Credentials.....	27
B.4 Orchestration Backend Setup (FastAPI).....	27
B.5 Sub-System Maintenance Guidelines.....	28
Appendix C – Code Listings.....	28
C.1 Introduction.....	28
C.2 Project Structure.....	29
C.3 Key Files and Main Functions.....	30
C.4 Important Code Snippets.....	32

1.0 Introduction

Modern web development is a big process. It involves several stages and several areas of knowledge, including requirements analysis, system design, implementation, and testing [2][15]. To build a full-stack web application, a developer has to understand what the user wants, design the system, write the frontend and backend code, design a database, and make sure everything runs together [2][15]. Experienced teams use software engineering methods and lifecycle models to manage all of this [2][15]. But for a single person, a student, or a small business, this process is expensive, slow, and complicated [2][15].

One of the biggest problems in software development is understanding what the user actually wants. Users usually describe their needs in natural language, or in informal documents, and their descriptions are often not complete, not clear, or ambiguous [2][8]. Traditional software engineering solves this problem through requirements engineering. Analysts convert the user's input into formal specifications before any coding starts [2][8]. When requirements are not clear, the final software often does not match what the user really wanted, even if the code itself works correctly [2].

AI tools have started to help with software development in the last few years. Coding assistants like GitHub Copilot [3] and Amazon CodeWhisperer [4] help developers by suggesting code inside the editor. Other tools, like Replit [6], let

users describe an application in natural language and produce something that runs. There are also no-code platforms like Bubble [9] that let people build apps with visual tools. The main goal of these tools is to make development faster and easier for people who do not know how to code [5][10].

These tools are useful, but they focus on code generation and not on understanding the requirements. Most AI coding assistants assume that the developer already knows what to build [5][7]. Application generators produce code in one shot, without checking that they understood the user correctly [7][11]. No-code platforms are limited in what they can build and often do not give the user full ownership of the code [9][10].

Another problem is that current tools do not really explain how they turn user input into code. It is hard to trace back from the generated code to the original requirements [8][11]. If the generated app does not match what the user wanted, it is not clear where the mistake happened [5][11]. So the user has to read, debug, and fix the code manually, which needs technical knowledge and takes away most of the benefit of automation [5][11].

To solve these problems, this project presents DeMaestro. DeMaestro is a role-based system that generates a full-stack web application directly from user requirements. It uses two LLM providers with different jobs: Google Gemini for analysis (extracting requirements, running the clarification loop, and writing the blueprint), and Anthropic Claude for code work (writing files, testing, debugging, and verifying) [14]. Each role runs one after the other, and every stage has to finish before the next one starts.

DeMaestro is designed around a requirements-first approach [2][8]. First, the user's input is turned into structured requirements. Then the system shows the user a Requirements Summary Document and asks for approval. Only after the user approves does the system start writing code. This way, the user has full traceability, and the system does not build the wrong thing based on a bad interpretation [2][8].

After the requirements are approved, the system produces a full source code project — frontend, backend, and database — and tests it inside a subprocess sandbox on the server. If the tests pass, the system deploys the project to Vercel with a fresh Neon Postgres database for that project. The user gets a live URL and a downloadable ZIP file with the full code [15].

DeMaestro is not made to produce production-ready software. Our goal was to give a solid starting point that a developer can extend. By producing consistent architecture, working code, and clear intermediate documents (the JSON contract, the Requirements Summary, the blueprint), DeMaestro reduces the gap between a user's idea and the first working version of the software. This is a project that fits into the scope of a bachelor-level software engineering degree [2][15].

The rest of the book is organized like this. Chapter 1 is this introduction. Chapter 2 reviews existing work. Chapter 3 describes DeMaestro at a high level. Chapter 4 describes the five phases of development. Chapter 5 explains the system architecture in detail. Chapter 6 describes the challenges we faced and how we

solved them. Chapter 7 presents our metrics and results. The references and three appendices (user guide, maintenance guide, and code listings_ are at the end.

2.0 Background and Related Work

Turning what a user wants into working software is a well-known problem in software engineering. In recent years, AI has started to help with this problem. Large Language Models (LLMs) can now write code and even help design systems, but they still cannot fully solve the gap between a user's idea and a working application. This chapter looks at the current landscape of AI-assisted development: real-time code assistants, natural language application generators, no-code platforms, and multi-agent LLM systems. For each type of tool, we describe what it does well and where it still falls short, especially in requirements engineering, traceability, and user validation. This shows why DeMaestro was needed.

2.1 AI-Based Code Assistants

Code assistants are one of the most common uses of LLMs today. Tools like GitHub Copilot [3] and Amazon CodeWhisperer [4] work inside the code editor and suggest code while the developer is typing. They can complete a function, suggest common patterns, or fill in boilerplate.

Studies show that these tools speed up development, especially for repetitive coding tasks [5]. But they are not autonomous. They assume the developer already knows what to build and how to structure it.

For a full-stack project, code assistants only help at the code level. They do not model the requirements, ask for confirmation from the user, keep track of what came from where, or check that the final system actually matches what was asked for [2][8].

2.2 Natural Language–Driven Application Generation

Some platforms try to go further and generate a whole application from a natural language description. The user types what they want and gets a program back. Replit, for example, has features that support this kind of flow [6].

These systems show that LLMs can be used for more than just code completion. But they have limits. Most of them work in one shot: the application is generated in one step, without asking the user to confirm anything [7][8]. This works for simple cases but breaks down when the requirements are complex or ambiguous. The result is often incomplete or inconsistent code.

The other issue is that these tools do not really show the user how their input was understood. There is no structured version of the requirements that the user can review, approve, or trace back to the generated code [8].

2.3 No-Code and Low-Code Development Platforms

No code and low code platforms take a different approach. They let users build applications by clicking and dragging in a visual interface. Bubble is one example [9]. These platforms are useful for simple use cases and for quick prototypes.

They also have well-known drawbacks. They limit customization and do not scale well for complex applications [10]. In many cases the user cannot even export the source code, which locks them into the platform. They also do not support formal requirements engineering or validation, which are important for real software projects [2][8].

2.4 Multi-Agent LLM Systems

Recent research suggests that instead of using one big LLM for everything, it is better to split a complex task across several specialized agents. Wooldridge covers the foundations of multi-agent systems as a research area [12], and Xi et al. show more recently how LLMs can be organized into different roles that work together [14]. In this kind of design, one agent might plan, another might execute, and another might verify.

The advantage is that each agent has a focused prompt and a smaller context to worry about. This reduces "context degradation" and makes the whole system easier to debug. If something goes wrong, we can usually point to a specific agent.

Most of the existing research on multi-agent LLMs focuses on general reasoning tasks (math problems, question answering, and so on). Very few systems combine multi-agent orchestration with a strict requirements-first workflow and a real subprocess-verified deployment pipeline. This is the gap that DeMaestro fills.

3.0 General Description

DeMaestro is a web-based platform. It acts like a virtual software engineering agency. Its job is to close the gap between high-level, informal requirements and a real, working full-stack web application. It solves a specific problem in modern software engineering: how to turn a user's idea, described in prose or in a document, into an executable application without losing what the user really wanted.

3.1 System Objectives

The main goal of DeMaestro is to automate the end-to-end creation of full-stack software while keeping strict rules about requirements validation. Unlike a simple wrapper around a code generator, DeMaestro runs a multi-stage lifecycle:

1. **Requirement Extraction.** The user uploads a PDF requirements document (up to 10 MB) or types a natural language description. The system uses the Gemini agents to extract entities, relationships, and functional requirements, and puts them into a strict JSON contract.
2. **Ambiguity Resolution.** The system automatically detects missing parts (for example, "which database framework?" or "what user roles are needed?") and asks the user with a short multiple-choice question.

3. **Explicit Validation.** Before any code is written, the system builds a human-readable Requirements Summary Document. The user has to explicitly approve this document. This is the requirement-first step.
4. **Architectural Blueprinting.** After approval, the system turns the requirements into a strict architectural blueprint that lists the API routes, database schemas, and frontend components.
5. **Code Synthesis and Subprocess Verification.** The Claude execution agents produce the actual source code for the frontend (React, Tailwind) and backend (Python FastAPI). The centralized orchestrator then runs this code in an isolated OS-level subprocess. Any stack traces are piped back to the debugging agent so it can patch problems.
6. **Live Preview Deployment and Packaging.** After the code passes the sandbox tests, the system deploys the application to Vercel using the Vercel Deployments API, and it creates an isolated Neon Postgres database just for that project. The user can interact with the live app in the browser, and the code is also packaged as a downloadable ZIP.

3.2 Target User Identification

DeMaestro is not aimed at just one kind of user. We designed it to serve several groups:

- **Software Architects and Senior Developers.** People who design high-level systems and want to skip the boring boilerplate. DeMaestro produces the directory structure, the dependency setup, and the CRUD APIs, so the architect can jump straight to the interesting business logic.
- **Entrepreneurs and Small Businesses.** People with an idea but with limited engineering budget. They get a working MVP that they can hand to a developer for further work.
- **Software Engineering Students and Junior Developers.** Students who are learning full-stack development and want to see how abstract requirements turn into real files, real schemas, and real endpoints. The intermediate artifacts (the JSON contract, the Summary, the blueprint) are useful teaching material.

3.3 The Agent Architecture: Centralized Orchestrator Multi-Agent Pattern

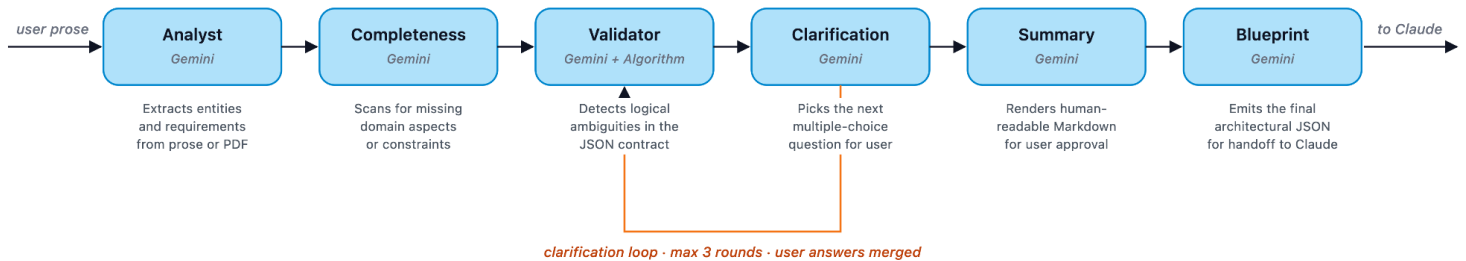
Early in the project we tried to use one single LLM for everything — parsing the prompt, planning, writing the code, and testing it. It did not work. The model would forget parts of the requirements halfway through the generation and produce files that did not match the earlier files. This is the context window degradation problem in practice.

To solve this, we designed DeMaestro around a Centralized Orchestrator Multi-Agent Pattern. Instead of one big model doing everything, the work is split across about a dozen specialized agents. Two centralized Python orchestrators run the flow and act as the deterministic part of the system.

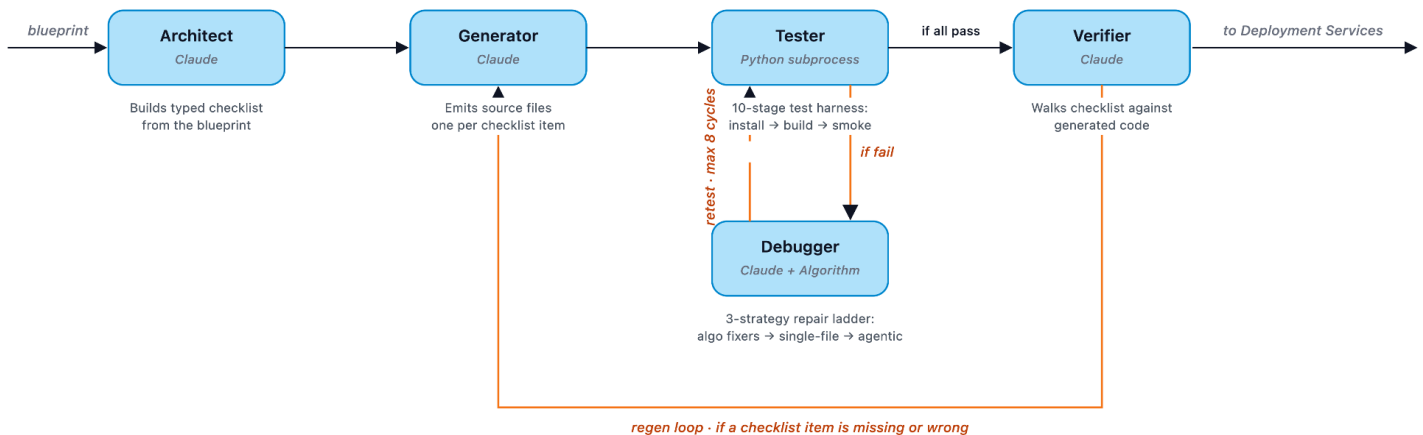
The agents are split into two logical groups:

- **Analytical Orchestrators (Gemini-Powered).** On the Gemini side we have six

agents. The Analyst extracts requirements from the prose or PDF. The Completeness checker looks for missing parts. The Validator detects ambiguities. The Clarification agent picks the next multiple-choice question to ask the user. The Summary agent writes the human-readable Markdown document. The Blueprint agent produces the final JSON blueprint that will be handed to Claude. Figure 3.1 shows this pipeline and the clarification loop.



- Execution and Debugging Agents (Claude-Powered).** On the Claude side we have five agents. The Architect turns the blueprint into a typed checklist. The Generator writes the source files one by one from that checklist. The Tester is a ten-stage test harness that runs the code inside a subprocess. The Debugger is a three-tier repair ladder that fixes broken code. The Verifier walks the final checklist against the generated files to make sure nothing was missed. Figure 3.2 shows this pipeline with the Test↔Debug loop and the Verifier regen loop.



We use two foundational LLM providers to balance analytical reasoning against raw coding capability:

- Google Gemini.** We chose Gemini for the analytical side because it is strong at following strict JSON schemas and it handles long context well, which matters for multi-page PDF parsing [16].
- Anthropic Claude.** We chose Claude for the code side because it is more reliable for granular, multi-file code writing, and it interprets error messages well when the Debugger sends stack traces back to it.

Splitting the roles this way, with a central orchestrator, is the main reason DeMaestro does not lose the user's original requirements by the time it reaches the code phase.

4.0 Development Process

The development of DeMaestro was iterative from day one. Generative AI is not deterministic, so a traditional waterfall approach did not really work — we needed to test, fail, and try again constantly. We used a five-phase development cycle so we could keep the AI models under control at each stage.

4.1 Phase 1: Research and Planning

We started by testing the available LLMs to see which ones could actually generate full-stack code. We tried Anthropic's Claude and Google's Gemini with the same natural language requirements. During this phase, a few things became clear:

- Using a single model for the whole pipeline did not work. The generated file structures were inconsistent and sometimes hallucinated entirely.
- We needed core features: PDF and natural language parsing, an interactive clarification loop, and a mandatory user approval gate.
- We decided on the Orchestrator Multi-Agent split — Gemini for analysis and Claude for code — that we still use today.
- In parallel we wrote our non-functional requirements (NFRs), including the 60-second limit for the requirements submission phase and the 5-minute limit for full code generation and packaging.

4.2 Phase 2: Architecture Design

After the research phase, we designed a decoupled architecture with clearly separated layers. The main decisions were:

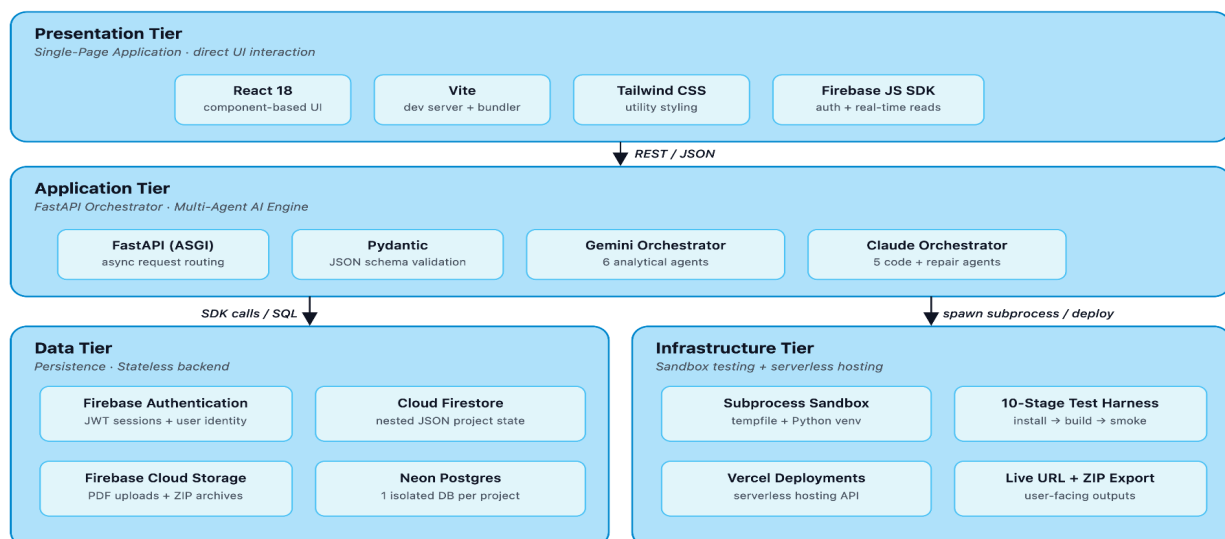


Figure 4.1 shows the resulting layered architecture.

- **Frontend.** React with Vite for a fast, state-driven interface. We needed something that could handle real-time chat interactions and long generation waits without freezing.
- **Backend.** Python with FastAPI. We picked FastAPI over Flask because it supports async natively (ASGI), and because Pydantic integration was going to be important for validating the JSON blueprints coming back from Gemini.
- **Database and Persistence.** Google Firebase (Authentication, Firestore, and Cloud Storage). This keeps the FastAPI backend stateless and gives us per-user project isolation for free. We also added Neon Postgres later as a per-project transactional database, provisioned only when a project is deployed.
- **Execution and Deployment Infrastructure.** An OS-level subprocess sandbox so we could safely run the AI-generated code before showing it to the user, plus a serverless Vercel deployment pipeline to host the final application in the cloud [17].

4.3 Phase 3: The Development Phase

Development happened in parallel across the stack. We split the work by area:

- **Backend Development (FastAPI):**
 - REST API endpoints for file upload, requirements processing, and project retrieval.
 - Integration layers for the Gemini API and the Claude API.
 - Strict Pydantic models for the "Hand-off Contract" — the JSON blueprint that goes from Gemini to Claude.
 - File packaging logic that creates per-project directory structures and compresses them into ZIP archives.
- **Frontend Development (React):**
 - Responsive components using Tailwind CSS.
 - The drag-and-drop PDF upload zone and the chat interface for the clarification loop.
 - A Markdown renderer for showing the Requirements Summary Document.
 - Loading skeletons, toast notifications, and progress indicators for the (sometimes long) generation window.
- **Subprocess Environment and Deployment Infrastructure:**
 - Python orchestration scripts that use `tempfile.mkdtemp` to create an isolated directory per project.
 - Subprocess-based sandboxing that spins up a fresh `python -m venv` for each project.
 - A serverless deployment client for the Vercel Deployments API.
 - Error-handling code that captures stack traces from the subprocess and pipes them back to the Debugger agent.

4.4 Phase 4: Integration and Testing

Once each layer worked on its own, we started wiring them together. The tricky part was making sure the async communication between the React client, the FastAPI orchestrator, Firebase, the multi-agent ecosystem, and the subprocess sandbox all worked without deadlocks or lost messages.

- **Unit Testing:** We deliberately sent malformed AI responses to the Pydantic validation layers to make sure the backend caught the exceptions and forced the orchestrator to retry, instead of writing bad data to Firestore.
- **Integration Testing:** We ran the whole pipeline end-to-end using different PDFs and natural language prompts, and we measured whether the generated React frontend and FastAPI backend actually matched each other (route names, data types, and so on).
- **Client Interface Feedback:** We showed the interactive prototype to peer software engineering students. Their feedback surprised us: the biggest problem was not the AI's answers, it was the way we asked the clarification questions. Open-ended text questions caused what we started calling "user fatigue" — students would abandon the flow after two or three questions.

4.5 Phase 5: Refinement and Optimization

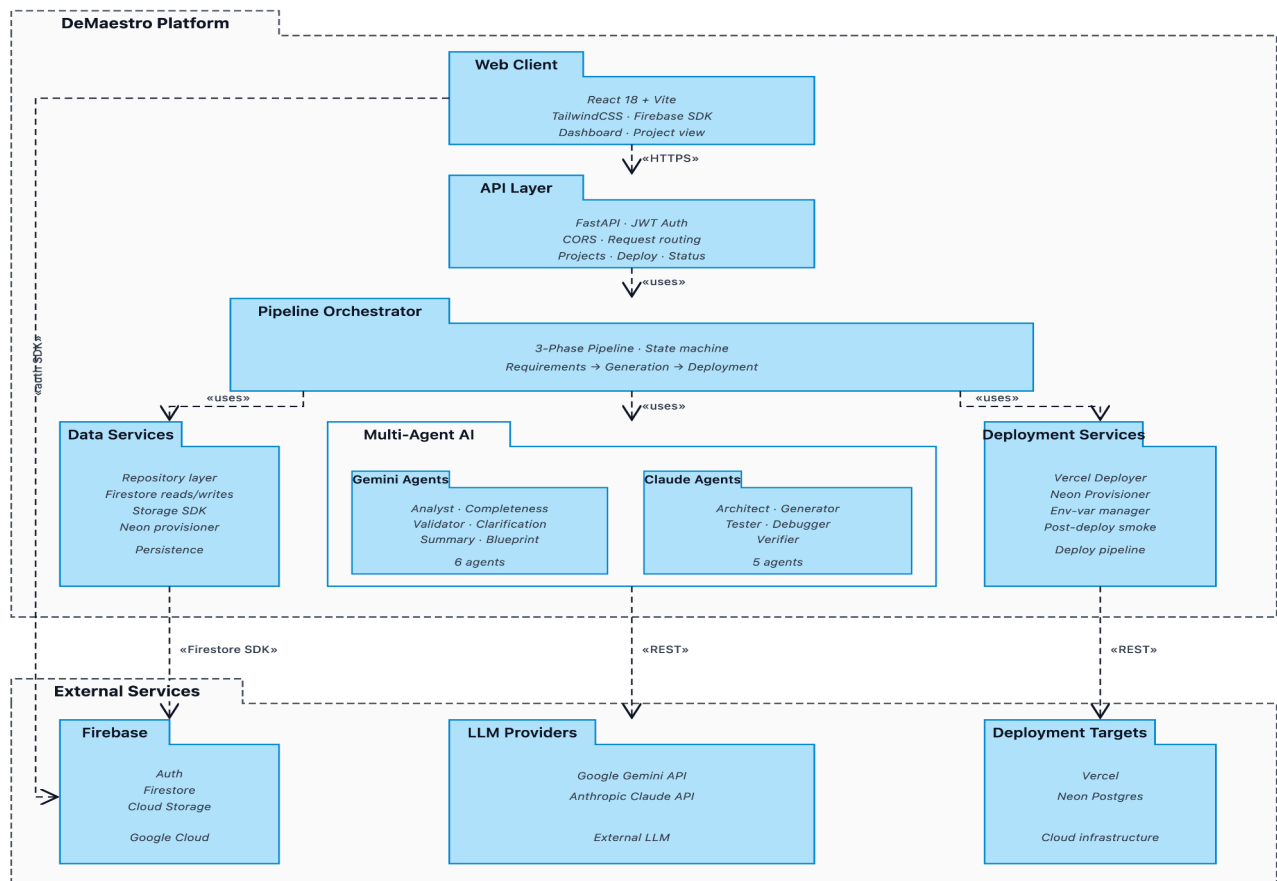
Based on the testing and the peer feedback, we made several changes:

- **Multiple-Choice Clarification.** To fix the user fatigue problem, we changed the clarification flow from open text to multiple-choice buttons. This was a small change in the UI but a big change in the user experience. Completion rates went up right away.
- **Prompt Engineering.** We tuned the system prompts for both Gemini and Claude. We were spending too many tokens on filler text at first, and the generated code was sometimes vague. Better prompts reduced token usage and gave us more precise output.
- **Error Handling.** We improved the frontend error boundaries so API timeouts and network failures show a friendly troubleshooting message instead of a broken page.

5.0 System Architecture

The DeMaestro architecture is designed to squeeze deterministic, verifiable behavior out of a set of non-deterministic AI models. To do this, the system keeps the user interface completely separate from the AI processing logic, and it uses a strict five-layer pipeline. Communication between the layers is one-way and always goes through the central orchestration server.

5.1 System Package Diagram



The Package Diagram (Figure 5.3) groups the codebase into five architectural layers inside the DeMaestro Platform container, plus three external service groups. The Activity Diagram (Figure 5.2) shows how a single project flows through those layers over time. The following subsections describe each layer.

5.1.1 The Frontend Layer (Web Client)

The frontend of DeMaestro is a Single-Page Application (SPA) built with React 18, Vite, and Tailwind CSS. We used a component-based structure, so each piece of the UI is a reusable module.

- **State Management.** AI generation is a long-running task, so the frontend had to keep the UI responsive during long waits. We used React Hooks together with a manual polling pattern (a “useEffect” with a “pollNonce” counter that forces the polling loop to restart after redeploy actions). This was simpler than pulling in a full state library.
- **Component Design.** The interface has a drag-and-drop upload zone, a chat

window for the clarification loop, and a Markdown renderer for the Requirements Summary Document.

- **Communication.** The frontend talks to the FastAPI backend through REST calls for pipeline operations, and directly to Firebase (using the Firebase JS SDK) for authentication and real-time project state updates. It does not talk to Gemini or Claude directly, every LLM call goes through the backend.

5.1.2 The Backend Orchestration Layer

The backend is written in Python 3.11 with the FastAPI framework. It is the central brain of the platform. It routes requests, runs the pipeline, and enforces schema validation.

- **Asynchronous Operations.** We picked FastAPI mainly for its native ASGI support. The server can handle many AI requests at the same time without one long generation blocking the others.
- **JSON Schema Validation.** Generative AI often produces slightly malformed output. We use Pydantic to define strict schemas for every intermediate JSON. If the AI returns something that does not fit, Pydantic raises an exception, the backend catches it, and the AI is asked to try again.
- **Workspace Management.** For every project the backend creates an isolated directory, writes the generated code into it, and later compresses it into a ZIP file for download.

5.1.3 The Data and Persistence Layer

DeMaestro uses Firebase together with Neon Postgres. This combination keeps the FastAPI backend stateless.

- **Firebase Authentication.** Handles registration and login, and issues JWT tokens that we use to authenticate every API call. Each user's projects are isolated.
- **Cloud Firestore.** A NoSQL document database. We use it for the nested JSON structures that the AI produces — raw user prompts, structured requirement contracts, and final blueprints.
- **Cloud Storage.** Stores the original PDF uploads and the final ZIP archives with the generated code.
- **Neon Postgres (per-project).** For every deployed project, DeMaestro creates a separate Postgres database on Neon, sets up its schema from the blueprint, and injects the connection string into Vercel's environment variables. Each generated app has its own database with real isolation from every other project.

5.1.4 The AI Agent Engine Layer

To avoid context degradation, DeMaestro replaces single-model generation with a Centralized Orchestrator Multi-Agent Pattern.

- **Analytical Orchestrators (Gemini-Powered).** The Gemini API powers the analysis side of the pipeline: parsing the initial prompt, running the clarification loop, and producing the JSON blueprint.
- **Execution and Debugging Agents (Claude-Powered).** The Claude Agent SDK powers the code side: the Architect (builds the checklist from the blueprint), the

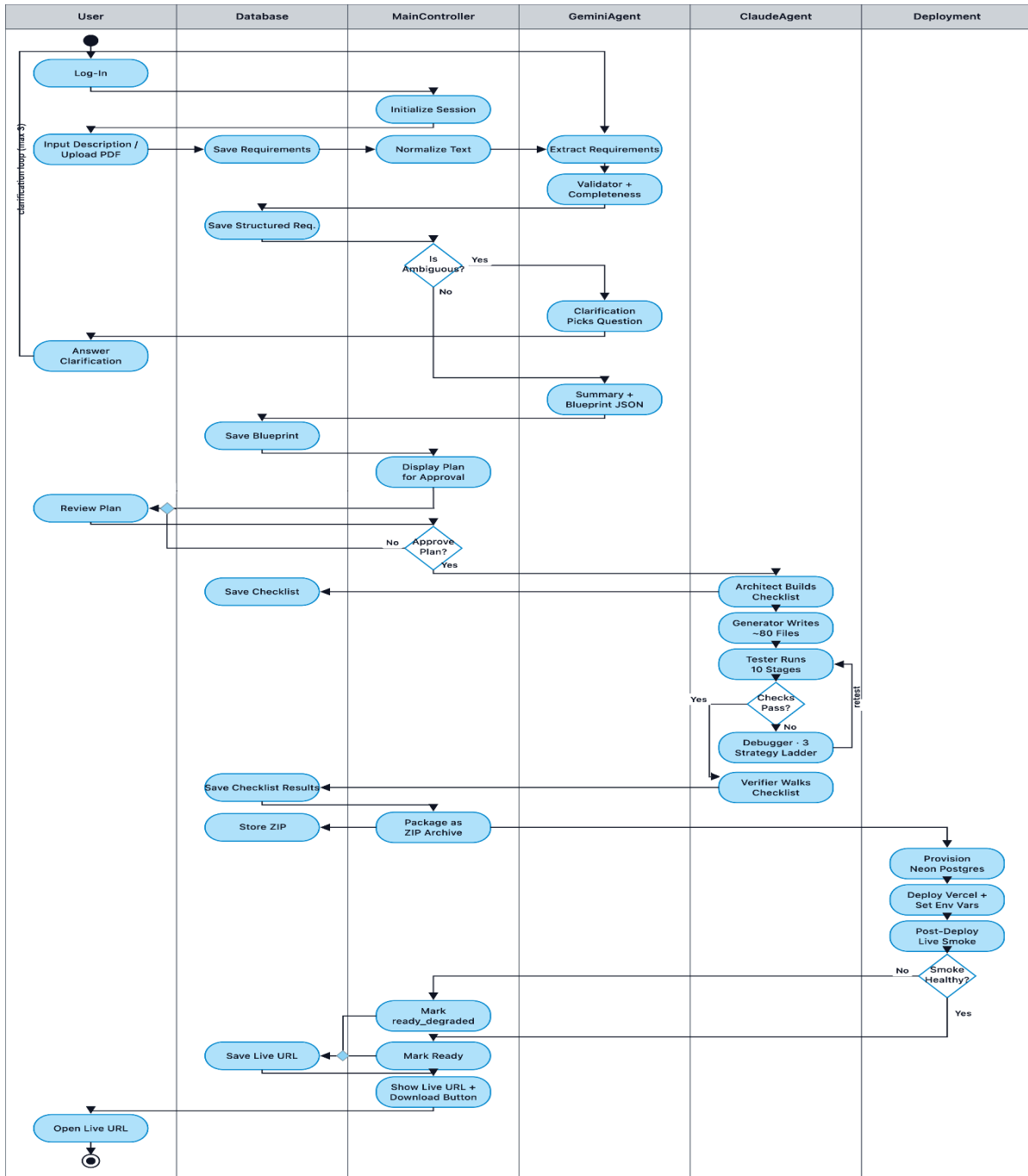
Generator (writes the source files), the Tester (runs the 10-stage test harness), the Debugger (runs the 3-strategy repair ladder), and the Verifier (walks the checklist against the generated code). The Debugger starts with 60 algorithmic Python fixers, then falls back to targeted single-file LLM regeneration, and finally to an agentic project-wide fix. The whole Test↔Debug cycle is capped at 8 attempts per project.

5.1.5 The Subprocess Execution and Cloud Deployment Layer

Because AI-generated code cannot be trusted blindly, we built a dedicated OS-level layer that tests the code and a serverless pipeline that hosts it.

- **Sandbox Testing (Subprocess):** For every generation, the backend creates a fresh isolated workspace with `tempfile.mkdtemp()`, spins up a `python -m venv`, installs dependencies in bounded worker threads, and runs a 10-stage test harness (install, lint, typecheck, boot, build, smoke, reachability, contract, FE↔BE match, browser smoke), all through Python subprocesses. Wall-clock timeouts protect the host from runaway processes.
- **Cloud Deployment (Vercel + Neon):** Once the sandbox passes, the Deployment Services push the whole repository to Vercel through its Deployments API, provision an isolated Neon Postgres database, wire up the environment variables, and run a live post-deploy smoke test against the deployed URL. A `ready_degraded` fallback is applied if the live smoke test fails but the application is still reachable.

5.2 System Flow



The operational workflow of DeMaestro is designed as a phased pipeline. The system strictly halts at the approval gate, which prevents "one-shot" AI hallucinations and ensures complete user traceability.

1. **Authentication and Input:** The user logs in with Firebase Authentication and starts a project by uploading a PDF requirements document (up to 10 MB) or entering a natural language description.
2. **Structuring Phase:** The FastAPI backend sends the text to the Gemini agent, which maps the unstructured data into a defined JSON requirements contract.
3. **The Clarification Loop:** The system analyzes the JSON contract. If variables are

missing (for example, unspecified database frameworks), the backend enters a clarification state. The frontend shows multiple-choice queries to the user. The loop repeats until the contract is logically complete.

4. **The User Approval Gate:** Once clarified, Gemini translates the JSON contract into a human readable Markdown document (the Requirements Summary). Execution halts here. The user must read and explicitly approve this document to continue.
5. **Blueprint Generation:** After user approval, Gemini produces the final architectural blueprint, mapping out API routes, database models, and frontend UI components.
6. **Code Synthesis and Subprocess Verification:** The blueprint is passed to the Claude execution agents, which write the frontend and backend files. The orchestrator creates a transient virtual environment, runs the code with Python subprocesses, and sends any stack traces to the Debugger for repair.
7. **Live Preview Deployment:** Once the generated code passes the sandbox test suite, the Deployment Services push the project to Vercel via its Deployments API and provision a per project isolated Neon Postgres database. A post deploy smoke test walks the live endpoints; if healthy, the project is marked ready and the live URL is stored in Firestore.
8. **Packaging and Export:** In parallel with the cloud deployment, the backend zips the workspace, uploads it to Firebase Cloud Storage with a signed URL, and stores the URL in Firestore. The dashboard shows both the Live URL and the Download ZIP buttons.

5.3 Implementation Overview

Chapter 5 has described DeMaestro at the design level: the package structure, the layered architecture, and the flow of a single project through the system. Chapter 5 does not include the file structure of the codebase, the main functions in each file, or the actual code, because those are implementation level details that would break the flow of the design description.

For readers who want to see how the design is implemented in code, all of this material is collected in **Appendix C – Code Listings**. Appendix C shows the full directory tree of the backend and the frontend, a table with the main function or class inside each important file, and six code snippets from the parts of the system that carry the most engineering weight (the Pydantic schema for the Hand off Contract, the Gemini Clarification agent, the Debugger strategy ladder, the subprocess sandbox runner, the Vercel deployer, and the frontend polling logic).

6.0 Challenges and Obstacles

Building an autonomous software generator meant fighting with unpredictable AI models every day. This chapter describes the main problems we ran into and how we solved them.

6.1 Enforcing Output Determinism from Generative AI

The very first end-to-end test of the pipeline crashed on us. Gemini returned the JSON requirements wrapped in a friendly sentence, something like "Here is your structured

requirements document:", and our parser blew up on the leading text. That was our first taste of structural hallucination. Even when we asked the model directly for pure JSON, it added filler text, or changed a `"snake_case"` key to `"camelCase"` halfway through the response. In an automated pipeline, one bad character is enough to kill everything.

- **The Solution:** We removed the AI's ability to chat freely at the backend level. We defined strict Pydantic models for the "Hand-off Contract" (the blueprint) inside our FastAPI server. Every prompt to Gemini demands this JSON structure. If the response does not match, Pydantic raises a validation exception, the backend catches it, and we automatically ask the AI to regenerate. This way, no malformed JSON ever reaches the code generation phase.

6.2 Host-Safe Execution via Subprocess Virtualization

In Phase A we said we needed to verify the generated code at runtime, not just look at it. But running AI-generated code directly on our main server would have been dangerous. A single infinite loop or a bad file write could take down the whole FastAPI service or eat all the memory:

- **The Solution:** We built an OS-level Virtual Environment Sandbox that runs each project inside its own transient directory tree. When Claude writes the code, the backend isolates the project with `"tempfile.mkdtemp()"`. Then it creates and activates a fresh Python virtual environment and runs the compilation check inside a bounded, non-blocking subprocess using Python's `"subprocess"` module. Wall-clock timeouts kill any runaway process, so the main server is protected and the project's dependencies stay inside its own directory.

6.3 The Debugger Strategy Ladder

The subprocess sandbox from Section 6.2 tells us when the generated code has failed, but not how to fix it. In the early versions, we tried the obvious approach: send the stack trace back to Claude and ask it to fix everything. This did not really work. Sometimes Claude fixed the reported error but broke something else in another file. Sometimes it rewrote the whole project and undid work that was already correct. Sometimes it kept making the same wrong fix over and over. So looping the LLM was expensive and often ended up worse than the starting point.

- **The Solution:** We built a three-tier escalation ladder that tries the cheapest, most targeted repair first and escalates only when it fails.
 - **Strategy 1 — Algorithmic Fixers (no LLM):** A library of 60 hand-written Python patches. Each one recognizes a specific failure signature we saw during early testing: a missing import, a mismatched Pydantic field type, an unregistered FastAPI route, a forgotten CORS header, a wrong Python `"venv"` path, an unused variable that trips lint, and so on. Each fixer runs in milliseconds. No LLM tokens are spent.
 - **Strategy 2 — Targeted Regeneration (single-file LLM):** If Strategy 1 leaves errors behind, the Debugger reads the stack trace, identifies the failing file, packages just that file plus its immediate imports, and asks Claude to regenerate the single file with the error message attached as context. The rest of the project stays untouched.

- **Strategy 3 — Agentic Holistic Fix (whole-project LLM):** Only when Strategy 2 also fails does the Debugger fall back to the most expensive option: sending the entire project plus the full test output to Claude in an agentic session, and letting it decide which files to modify. This strategy is capped at 5 invocations per project to prevent runaway cost.

Between each strategy the Tester re runs, and a project snapshot is captured before every fix so a bad patch can be rolled back instead of committed. The whole Test - Debug loop is capped at 8 cycles per project, if all three strategies exhaust their budget, the project is marked “**ready_degraded**” and shipped with the last known good snapshot instead of looping forever. This tiered escalation is the main reason DeMaestro's projects reach deployment at a higher rate than a naive LLM-loop baseline: cheap fixes handle the common case, and the expensive fix is used only for the rare hard case where it is really needed.

6.4 Serverless Cloud Deployment via Vercel

At first, we thought about running a live preview of each generated app on our own server. This turned out to be a bad idea. Every project would need an open network port on the host, a temporary Nginx reverseproxy configuration, and some way to prevent it from clashing with other running projects. This is a lot of moving parts, all of which are security risks and single points of failure on the orchestration server.

- **The Solution:** We removed the local networking problem completely by using the Vercel Deployments API. When a project passes the sandbox verification, a dedicated deployment controller packages the frontend and backend files and provisions an isolated Neon Postgres database for that project. Then it pushes the generated assets to Vercel's cloud, injects the Neon connection string as an environment variable, and starts a remote build. Vercel gives back a cloud managed, isolated deployment URL. We store this URL in Firestore and show it to the user. The user can open a live version of their app without any routing or container overhead on our infrastructure.

6.5 Lessons Learned

The project reached its main goal, closing the gap between an idea and a first working version. Along the way we learned some things we would take into a future version:

1. **The Necessity of Multi Agent Orchestration:** We tried the single-model approach for the first weeks of the project. It did not work, the model would drift, forget the schema, and hallucinate file paths in longer generations. The moment we split the work between Gemini and Claude, the end-to-end pipeline started actually finishing.
2. **API Latency and Cost.** The tiered Debugger ladder from Section 6.3 keeps the average cost low, but projects that reach Strategy 3 still use a lot of tokens, and the full Test-Debug loop can add several minutes, sometimes much more, to the generation time. For large applications with many models and routes, the total generation time can reach 10 to 15 minutes, which is the main reason we did not fully meet NFR6. If we had more time, we would work on two things. First, we would add a caching layer for recurring code blocks (like standard Firebase authentication setups) to cut down on token consumption. Second, we would parallelize the Generator so it emits independent files at the same time instead of one after

another, which should cut the total generation time significantly for larger apps.

3. **Database Migration Limitations:** DeMaestro today provisions a fresh database schema every time. If a user wants to update an existing project, for example, add a new field to a model, the system does not know how to generate a safe migration script (something like Alembic for Python). Supporting stateful, iterative generation would be a big next step: each new run would diff against the previous blueprint and produce migration steps instead of a fresh schema.

7.0 Project Metrics

In Phase A we defined a set of Non Functional Requirements (NFRs) to measure whether the system was actually working. During Phase B we focused our testing on two areas: system performance (latency) and code reliability. For readability, we restate the relevant NFRs inline when we evaluate them.

7.1 System Performance and Generation Time

Performance is important because a slow system loses users, no matter how good the output is.

- **Requirement Gathering (NFR5 - the clarification loop must finish in under 60 seconds):** The React client keeps state locally and only calls the Gemini API when there is a real ambiguity. During our tests, the whole clarification loop (up to 3 rounds) finished well under 60 seconds, so this NFR was met.
- **Full-Stack Synthesis Time (NFR6 — full generation and packaging must finish in 5 minutes).** This is the NFR we did not fully meet. In Phase A we thought 5 minutes would be enough, but the real generation time depends heavily on how many models, routes, and components the requirements ask for. For a small app (about 2 to 3 models and a handful of CRUD routes) the pipeline finishes in around 3 to 5 minutes, which meets the target. For a larger app (6 to 8 models, nested relationships, more than a dozen routes, and a richer frontend) the whole pipeline takes about 10 to 15 minutes end-to-end. Most of the extra time comes from the Test-Debug loop, a larger project has more integration points between the frontend and the backend, so the Debugger runs more repair cycles before all 10 test stages pass. To keep the user from thinking the system is stuck during long runs, the UI shows progress messages for each active stage (Generating Frontend Components, Synthesizing Backend Routers, Running Automated Integration Tests, and so on). Because the project state is written to Firestore after each stage, the user can close the browser during a long generation and come back later, the dashboard shows the live status of the project, and the user is notified when the generation finishes.

7.2 Code Reliability and Validity Rate

Our core success metric was the "Code Validity Rate", the percentage of generated projects that actually compile and boot without a runtime crash (NFR2, every project must compile; NFR3, every project must boot without crashing).

- **Before vs. After the Sandbox.** Early in Phase B, before the subprocess sandbox and the Debugger were integrated, the validity rate was around 40%. The most

common failures were mismatched API routes between the React frontend and the FastAPI backend, and unverified dependency imports. These were silent errors at runtime, the code looked fine, but it broke as soon as you tried to run it.

- **After the Multi-Agent Pattern and Sandbox.** Once we added the Orchestrator Multi Agent Pattern and the automated subprocess testing loop, the validity rate improved significantly. Because the Tester runs the code before deployment and the Debugger fixes the errors that come out of it, projects that reach the Vercel deployment step consistently boot and run. The NFR2 and NFR3 requirements are met by construction, the pipeline does not deploy a project that fails the sandbox.

7.3 Usability and Interface Effectiveness

We wanted to check whether the system was actually usable, not just working. We ran short usability sessions with peer software engineering students. Their early feedback was clear: open-ended chat with the AI was tiring and confusing. Students would get stuck on the second or third question and give up. After we changed the clarification UI to multiple-choice buttons, both user confidence and completion rates went up. This confirmed that the requirement-first methodology is only useful if the user actually finishes the requirement-gathering phase.

References

- [1] Brown, T. et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.
- [2] Sommerville, I., *Software Engineering*, 10th Edition, Pearson, 2015.
- [3] GitHub Copilot, product page.
- [4] Amazon CodeWhisperer, product page.
- [5] Vaithilingam, P. et al., "Expectations, Outcomes, and Challenges of Using AI-Based Code Generation Tools," CHI, 2022.
- [6] Replit, product website.
- [7] Chen, M. et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.
- [8] Nuseibeh, B., and Easterbrook, S., "Requirements Engineering: A Roadmap," 2000.
- [9] Bubble, product website.
- [10] Poppendieck, M., and Poppendieck, T., *Lean Software Development*, Addison-Wesley, 2003.
- [11] Kulkarni, C. et al., "Semantic Gaps in AI-Assisted Programming," ACM TOSEM, 2021.
- [12] Wooldridge, M., *An Introduction to Multi-Agent Systems*, 2nd Ed., Wiley, 2009.
- [13] Stone, P. et al., "Artificial Intelligence and Life in 2030," Stanford, 2016.
- [14] Xi, Z. et al., "The Rise and Potential of Large Language Model Based Agents," arXiv:2309.07864, 2023.
- [15] Pressman, R. S., and Maxim, B. R., *Software Engineering: A Practitioner's Approach*, 8th Ed., McGraw-Hill, 2014.
- [16] Google, "Gemini: A Family of Highly Capable Multimodal Models," Google AI Blog, 2023.
- [17] Humble, J., and Farley, D., *Continuous Delivery*, Addison-Wesley, 2010.

Appendices

Appendix A – User Guide

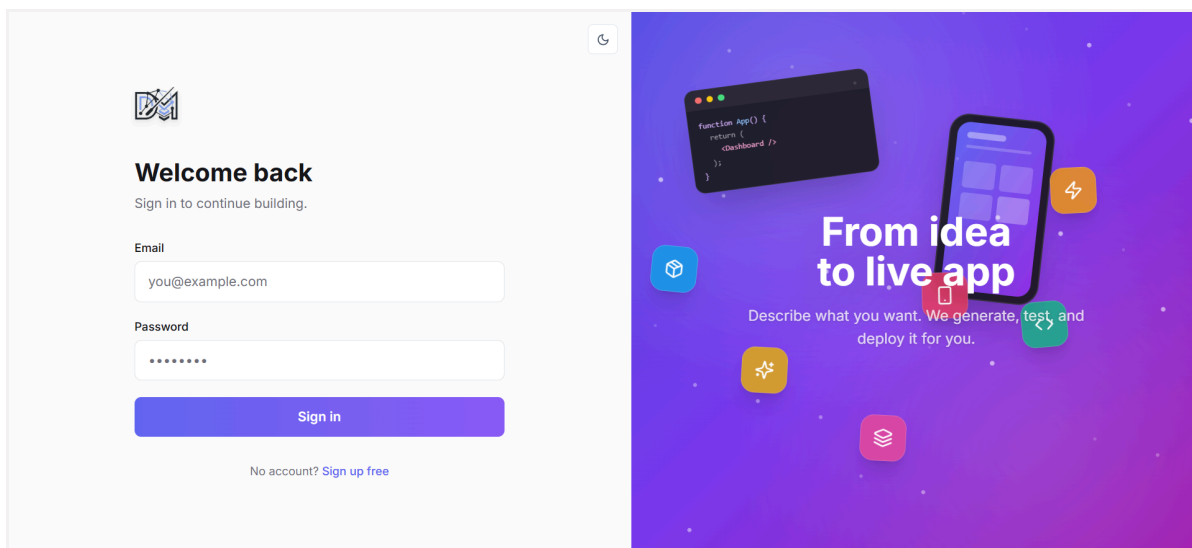
A.1 Introduction

DeMaestro is a web-based platform that turns your software idea into a running full-stack web application. You give it a natural language description or a PDF requirements document, and it produces React frontend code and FastAPI backend code, tests them, and deploys them for you. It is aimed at software engineers, students, and entrepreneurs who want to go from an idea to a live demo in minutes.

This guide walks through the full workflow, from creating an account to downloading the final source code.

A.2 Operating Instructions

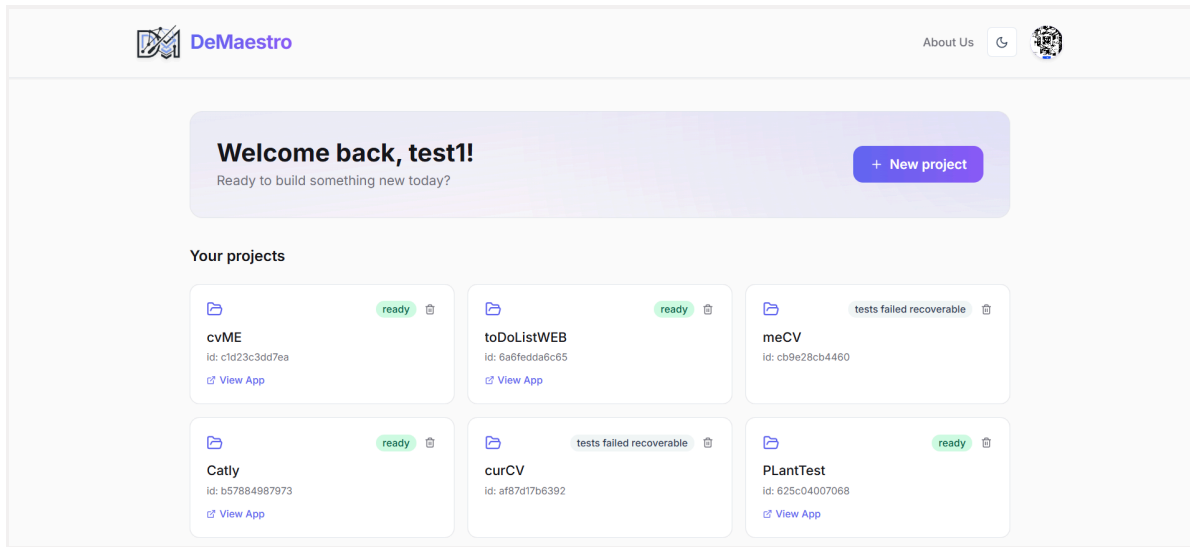
A.2.1 Sign-Up and Authentication Page



To use DeMaestro's features and save your projects, you need an account.

1. Open the DeMaestro web application in your browser.
2. Click **Signup** in the top navigation bar.
3. Fill in your email and password.
4. Click **Sign Up**.
5. If you already have an account, click **Login**, enter your credentials, and click **Login**. After successful login, you are redirected to the main dashboard.

A.2.2 The Main Dashboard



The dashboard is your workspace. It shows a grid of your saved projects.

- To open a past project, click **View** on any project card. You can see the blueprint, the original requirements, and the download links.
- To start a new one, click **+ New Project** at the top of the workspace.

A.2.3 The Requirements Input Page

Upon

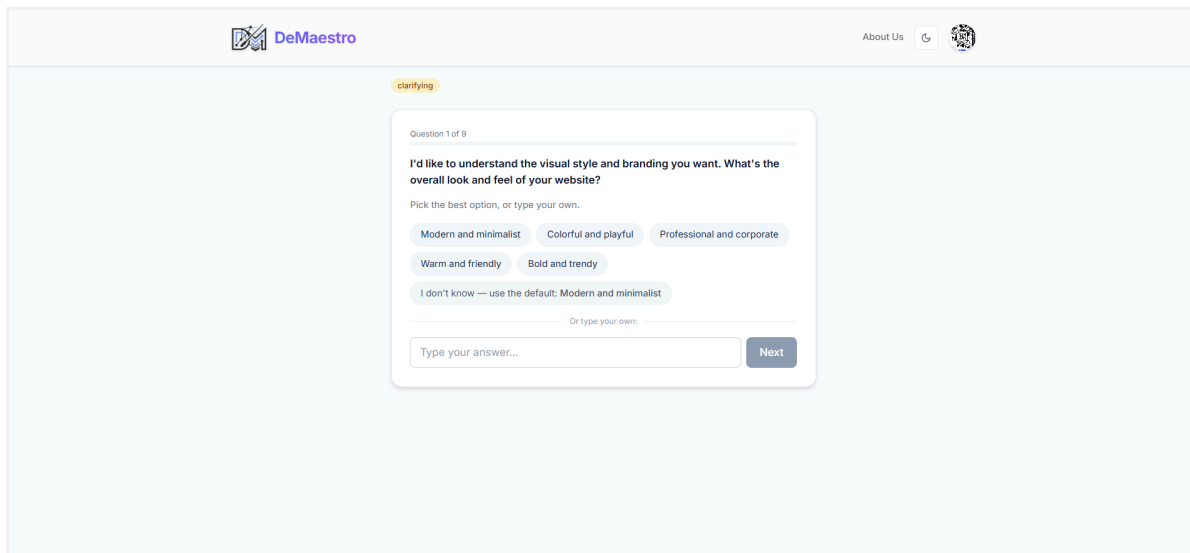
initiating a new project, you will be presented with the input interface where you define your software idea.

1. Upload Options:

- **Document Upload:** Drag and drop your requirements PDF (up to 10MB) into the designated upload area, or click to browse your local files.

- **Text Prompt:** Alternatively, use the text area to type a natural language description of your desired web application.
- 2. Once your file is attached or your text is entered, click the **Analyze Requirements** button. The system will transition to a loading state while the AI agent extracts the core entities and logic.

A.2.4 The Clarification Chat Loop

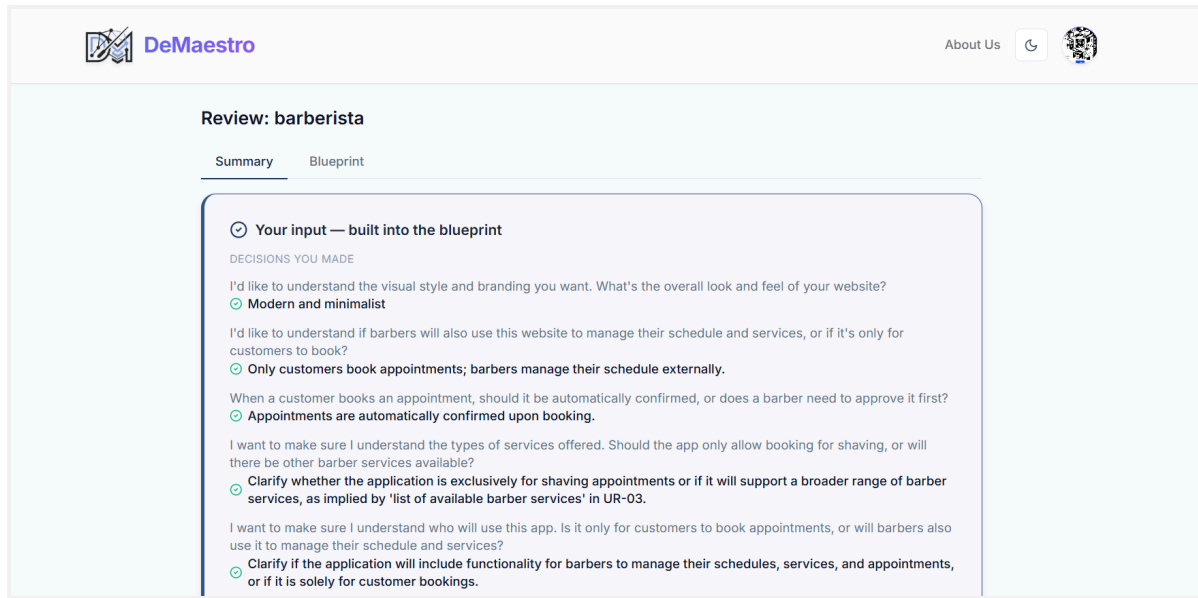


The screenshot shows the DeMaestro web application interface. At the top, there is a header with the DeMaestro logo and an 'About Us' link. Below the header, a yellow label 'clarifying' is visible. The main content area displays a chat bubble for 'Question 1 of 9'. The question is: 'I'd like to understand the visual style and branding you want. What's the overall look and feel of your website?'. Below the question, it says 'Pick the best option, or type your own.' There are six button options: 'Modern and minimalist', 'Colorful and playful', 'Professional and corporate', 'Warm and friendly', 'Bold and trendy', and 'I don't know — use the default: Modern and minimalist'. At the bottom of the chat bubble, there is a text input field labeled 'Type your answer...' and a 'Next' button.

DeMaestro does not guess when something is missing. If your input does not include something important (for example, a preferred database or missing user roles), the system stops and asks.

1. A chat window shows a specific question from the AI (for example, "Which database would you like for the authentication system?").
2. Below the question, you see multiple-choice buttons with the supported options.
3. Click the option that fits your needs.
4. If there are more ambiguities, the system asks follow-up questions. Once everything is clear, the system moves you to the Approval Gate.

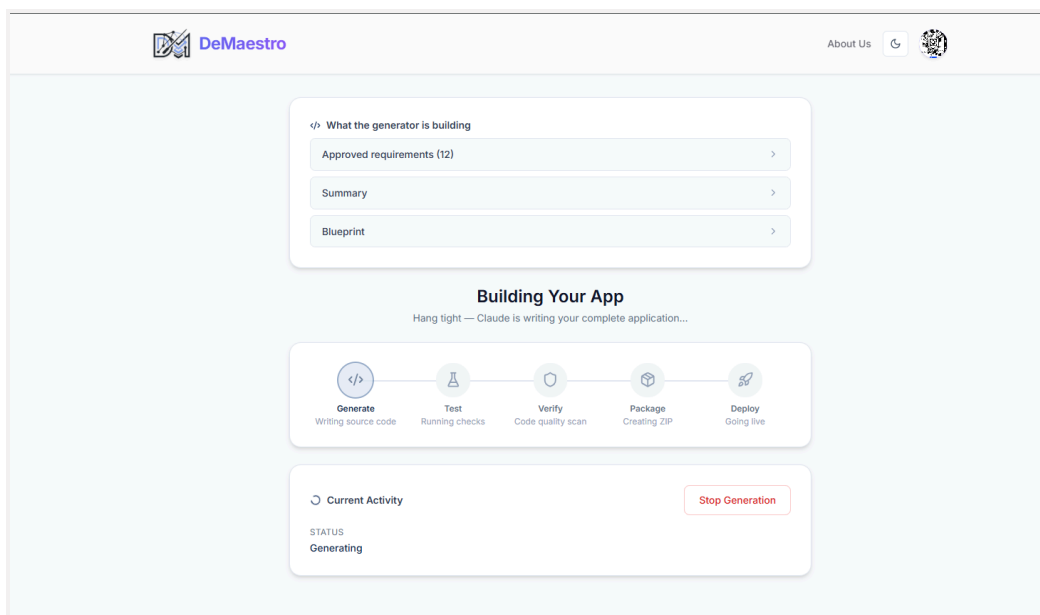
A.2.5 The User Approval Gate



Before any code is written, DeMaestro shows you what it plans to build.

1. The screen shows a formatted Requirements Summary Document (rendered in Markdown).
2. Read it carefully. It lists the frontend components, backend API routes, and database schema that the system will build.
3. If everything looks right, click **Approve and Generate**.
4. Note: once you approve, the blueprint is locked and the code synthesis phase starts.

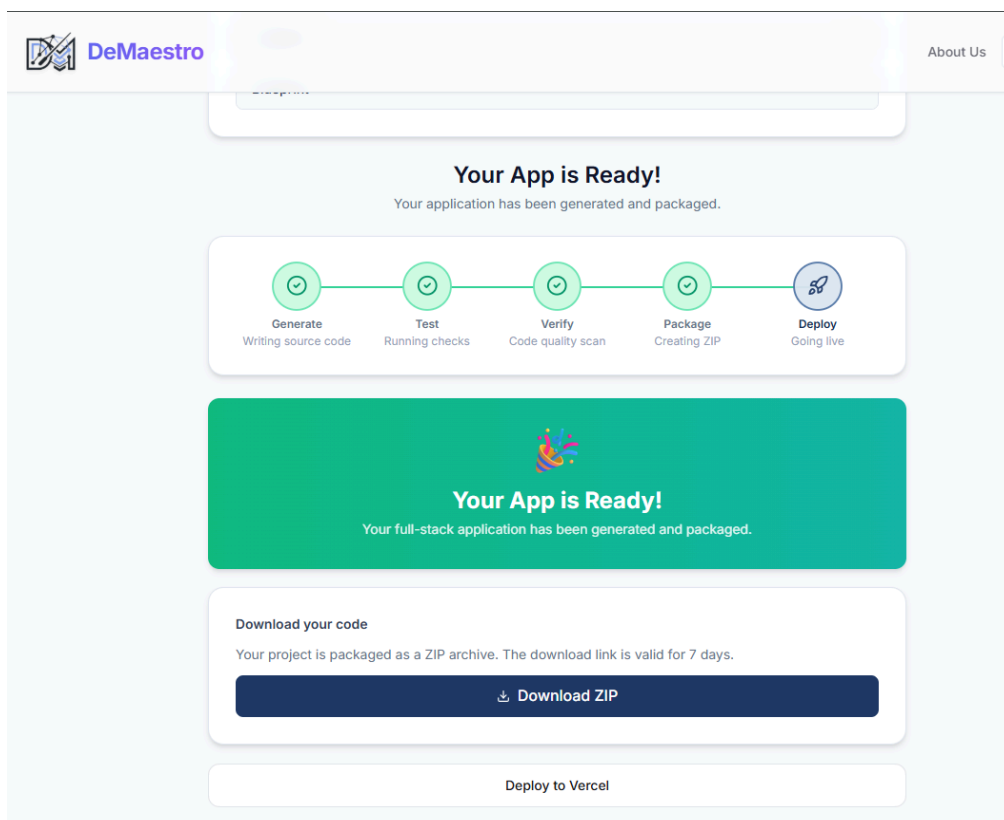
A.2.6 Generation and Sandboxing Status



Full-stack synthesis, virtualization, and testing take a few minutes. During this time you see a progress screen.

1. A loading animation shows the message "Synthesizing code and initializing virtual environment..."
2. A progress indicator shows the active stage (Generating Frontend Components, Synthesizing Backend Routers, Running Automated Integration Tests, and so on).
3. The process usually takes between 3 and 15 minutes, depending on how complex the app is and how many debug cycles the system needs. Simple CRUD apps finish quickly; larger apps with many models and routes take longer.

A.2.7 Live Preview and Download Page



Once generation and cloud testing finish, you are automatically taken to the Export page.

1. **Live Preview.** In the middle of the interface, an embedded iframe renders your app. This is not a local sandbox — it is a real Vercel deployment with a unique URL. You can click through the app and test it directly in this window.
2. **Export Options.** Below the preview:
 - **Download Blueprint** — the JSON file that describes the architecture.

- **Download Source Code (ZIP)** — the full compressed project with all React, FastAPI, and configuration files, ready to run locally.

A.3 Navigation and Workspace Management

A.3.1 The Navigation Bar

The top navigation bar remains persistent across all authenticated screens, providing quick access to essential areas:

- **Dashboard:** Returns you to the grid view of your saved projects.
- **New Project:** Instantly aborts the current view and opens a fresh upload screen.
- **Logout:** Securely terminates your active session.

A.3.2 Real-Time Notifications

DeMaestro uses toast notifications (small pop-ups in the corner of the screen) to give you immediate feedback. You will see them for successful uploads, invalid file formats, network timeouts, and any other important events.

Appendix B – Maintenance Guide

B.1 Introduction

This Maintenance Guide provides the necessary technical documentation for future DevOps engineers or system administrators responsible for maintaining the DeMaestro platform's core orchestration layer. Because DeMaestro functions as an autonomous generation utility, its infrastructure relies on subprocess sandboxing, per-project virtual environments, and multi-agent API management. This guide outlines the specific hardware prerequisites, environment configurations, and step-by-step instructions required to run and maintain the DeMaestro orchestration server and its sandbox environment.

B.2 System Requirements

B.2.1 Hardware Requirements

Because DeMaestro utilizes localized OS-level virtualization to execute and verify code, the host server must maintain sufficient memory allocation to prevent sub-process timeouts during active generation.

- **Processor:** Multi-core CPU (Intel i5 / AMD Ryzen 5 or equivalent or higher).
- **Memory:** Minimum 8 GB RAM (16 GB recommended to handle multiple concurrent multi-agent processing pipelines).
- **Storage:** 5 GB of free disk space (required primarily for virtual environment caching, file asset structuring, and python library requirements).

B.2.2 Software Requirements

The backend orchestration server runs directly on the host machine OS environment. The following dependencies must be installed natively prior to initialization:

- **Python:** Version 3.11 or higher (strictly required for FastAPI core and Pydantic schema validation layers).
- **Node.js & npm:** Version 18.x or higher (required on the host system to run local build/verification checks).
- **Package Manager:** **pip** (Python package manager).

B.3 Configuration and API Credentials

The DeMaestro orchestration backend relies on several external cloud services. The system administrator must provision the following credentials:

1. **Google Firebase:** A Firebase project must be active with Authentication, Firestore, and Cloud Storage enabled. Generate a Firebase Admin SDK private key (JSON file) from the project settings.
2. **Google Gemini API Key:** Required for the six analytical Gemini agents (Analyst, Completeness, Validator, Clarification, Summary, and Blueprint).
3. **Anthropic API Key:** Required for the five Claude execution and debugging agents (Architect, Generator, Tester, Debugger, and Verifier).

The “**.env**” file in the root directory of the backend must adhere to the following configuration structure:

GEMINI_API_KEY=your_gemini_key_here

ANTHROPIC_API_KEY=your_claude_key_here

B.4 Orchestration Backend Setup (FastAPI)

The backend orchestration server must be executed within an isolated Python virtual environment to prevent dependency conflicts with the host system.

1. Navigate to the backend directory on your server.

Create and activate a Python virtual environment:

```
python3 -m venv venv
```

2. `source venv/bin/activate`
3. Install the required Python packages (including FastAPI, Uvicorn, Pydantic, and your LLM engine clients):
`pip install -r requirements.txt`
4. Place the “**firebase-admin.json**” service account file strictly inside the “**config/**” directory.
5. Start the FastAPI server using Uvicorn:
`uvicorn main:app --reload`

The orchestration server is now active. System administrators can view the automatic OpenAPI documentation and test routes at <http://localhost:8000/docs>.

B.5 Sub-System Maintenance Guidelines

B.5.1 Modifying the AI Architectural Blueprint

If future maintainers wish to expand the capabilities of DeMaestro (e.g., adding a generation layer for automated UI testing frameworks), the "Hand-off Contract" must be updated. You **must** update the Pydantic schemas located in the “**models.py**” file. If the Pydantic schemas do not perfectly map to the prompt instructions given to Gemini, the backend validation pipeline will reject the AI's output and trigger an infinite regeneration loop.

B.5.2 Troubleshooting the Serverless Live Preview

The Live Preview deployment relies on the Vercel Deployments API for serverless cloud hosting. If the generated applications fail to render in the frontend iframe, administrators should first verify that the host server has outbound network access to Vercel's API endpoints. Additionally, check the Python orchestration logs for local subprocess compilation errors that may have prevented the deployment phase from initiating. If the source code was successfully pushed but the iframe fails to load, administrators should inspect the remote Vercel project build logs to identify any cloud-side dependency or routing failures.

Appendix C – Code Listings

C.1 Introduction

This appendix collects the file structure and the most important code from the DeMaestro project. The goal is to show the actual implementation for the parts of the system that carry the most engineering weight - schema validation, agent logic, subprocess sandboxing, cloud deployment, and frontend polling. Each listing has a short introduction that explains what the code does and why it was written this way.

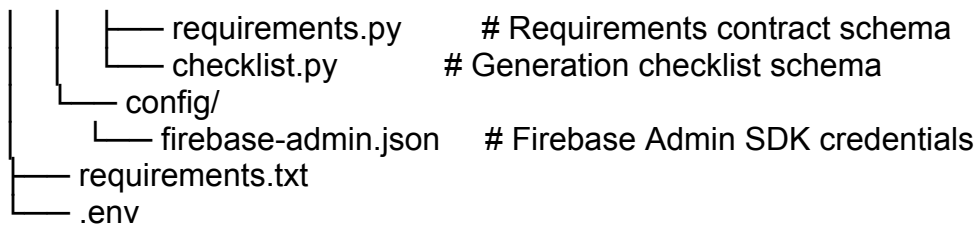
The listings are kept short on purpose. We picked only the pieces that show the main design decisions, not every helper function or utility. If you want to see the full code, please check the repository referenced in the project's **"README.md"**.

C.2 Project Structure

The DeMaestro code is split into two main parts. The backend is a Python project that runs the FastAPI server and the multi-agent pipeline. The frontend is a React project that talks to the backend and shows the user interface. Each side has its own directory tree.

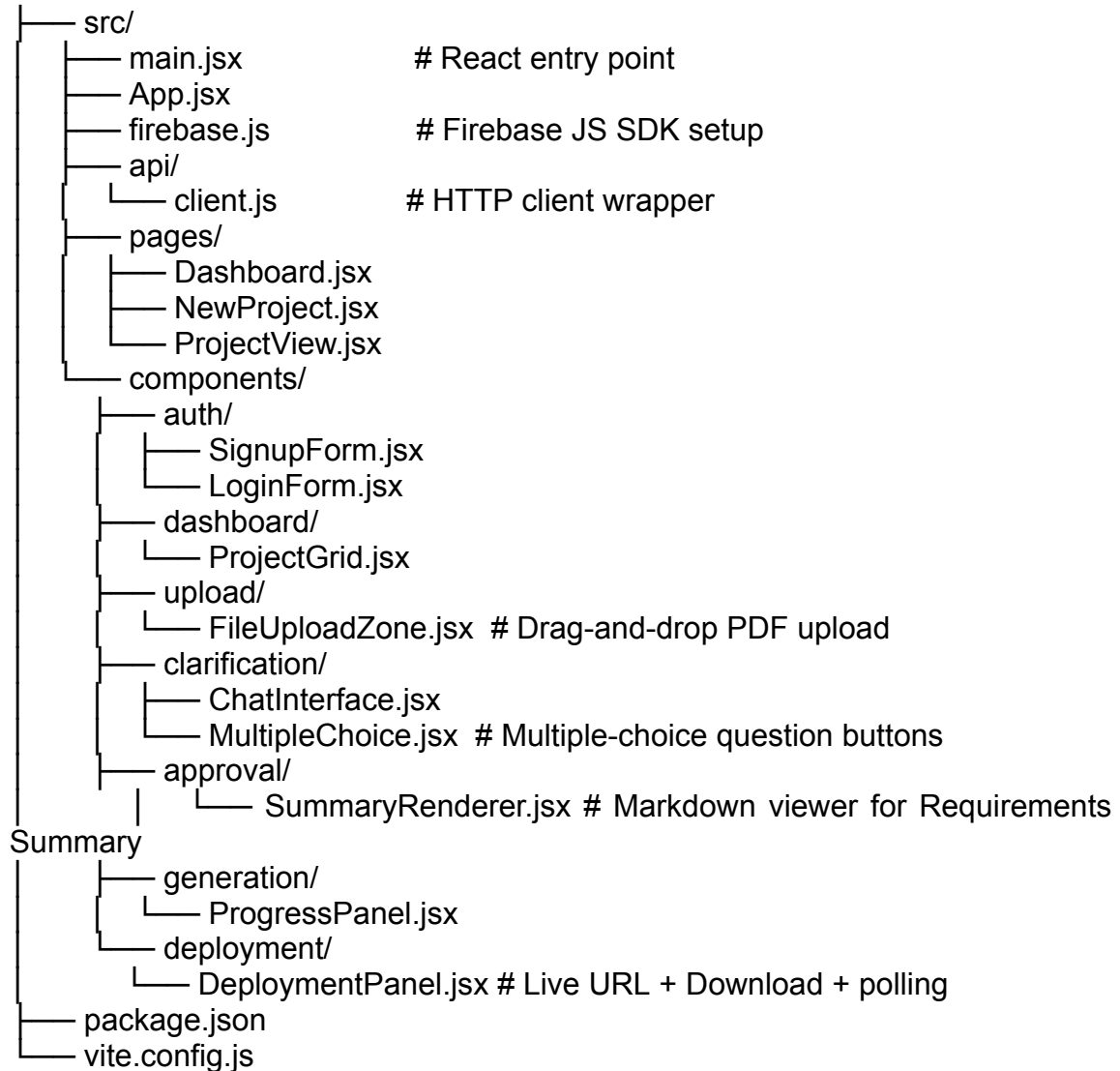
C.2.1 Backend Structure

```
demaestro/backend/
├── app/
│   ├── main.py           # FastAPI application entry point
│   └── api/
│       ├── auth.py       # Firebase auth endpoints
│       ├── projects.py   # Project CRUD endpoints
│       ├── deployment.py # Deploy trigger + status endpoints
│       └── clarifications.py # Clarification loop endpoints
│   └── ai/
│       ├── gemini/
│       │   ├── client.py # Gemini API wrapper
│       │   └── agents/
│       │       ├── analyst.py
│       │       ├── completeness.py
│       │       ├── validator.py
│       │       ├── clarification.py
│       │       ├── summary.py
│       │       └── blueprint.py
│       └── claude/
│           ├── client.py # Claude Agent SDK wrapper
│           └── agents/
│               ├── architect.py
│               ├── generator.py
│               ├── tester.py
│               ├── debugger.py
│               └── verifier.py
│   ├── orchestrators/
│   │   ├── requirements_orchestrator.py
│   │   └── generation_orchestrator.py
│   ├── services/
│   │   ├── firebase_service.py # Firestore + Storage + Auth SDK
│   │   ├── neon_provisioner.py # Per-project Postgres DB creator
│   │   ├── vercel_deployer.py # Vercel Deployments API client
│   │   ├── sandbox_runner.py  # Subprocess sandbox
│   │   └── zip_packager.py     # ZIP archive builder
│   ├── models/
│   │   └── blueprint.py       # Pydantic schema for the Hand-off Contract
```



C.2.2 Frontend Structure

demaestro/frontend/



C.3 Key Files and Main Functions

Below is a list of the files that carry most of the work. For each file we mention the main function or class inside it, and what it does. These are the files a new developer or a future maintainer needs to look at first.

- **app/main.py** — `create_app()` — Starts the FastAPI app, adds the routers, and loads the middleware.
- **app/api/projects.py** — `create_project()`, `get_project()` — Handles project CRUD. Writes to Firestore under `/users/{uid}/projects/{projectId}`.
- **app/api/deployment.py** — `trigger_deploy()`, `get_status()` — Starts a deployment and returns the current status for the frontend to poll.
- **app/api/clarifications.py** — `submit_answer()` — Takes the user's answer to a clarification question and re-runs the Validator.
- **app/models/blueprint.py** — `Blueprint` (Pydantic model) — The strict schema that Gemini has to produce. If the response does not fit, we trigger a retry.
- **app/ai/gemini/agents/analyst.py** — `extract_requirements(text)` — Turns user prose or PDF text into a structured JSON requirements contract.
- **app/ai/gemini/agents/validator.py** — `validate_contract(contract)` — Looks for ambiguities. Returns a list of missing or unclear fields.
- **app/ai/gemini/agents/clarification.py** — `pick_next_question(contract, missing)` — Picks the single most important multiple-choice question to ask the user.
- **app/ai/gemini/agents/blueprint.py** — `generate_blueprint(contract)` — Produces the final architectural JSON blueprint after the user approves the summary.
- **app/ai/claude/agents/architect.py** — `build_checklist(blueprint)` — Turns the blueprint into a typed checklist, one item per file to be produced.
- **app/ai/claude/agents/generator.py** — `generate_file(item)` — Writes one file at a time, based on a single checklist item.
- **app/ai/claude/agents/tester.py** — `run_all_stages(project_dir)` — Runs the 10-stage test harness inside the subprocess sandbox.
- **app/ai/claude/agents/debugger.py** — `repair_project(dir, test_result, cycle)` — The 3-strategy repair ladder (algorithmic → single-file → agentic).
- **app/ai/claude/agents/verifier.py** — `verify_checklist(dir, checklist)` — Checks that every checklist item was actually produced correctly.
- **app/services/sandbox_runner.py** — `SandboxRunner.run_test_stage(dir, stage)` — Runs one test stage in a subprocess with a wall-clock timeout.
- **app/services/vercel_deployer.py** — `deploy_project(project_id, files, neon_url)` — Uploads the project to Vercel and polls until READY.
- **app/services/neon_provisioner.py** — `create_project_db(slug)` — Creates a fresh Neon Postgres database and returns the connection string.
- **app/orchestrators/generation_orchestrator.py** — `run_generation_pipeline(project_id)` — Coordinates Architect → Generator → Tester → Debugger → Verifier.

- **frontend/src/components/deployment/DeploymentPanel.jsx** — **DeploymentPanel** (React component) — Polls the deployment status every 4 seconds and shows the live URL when it is ready.
- **frontend/src/components/upload/FileUploadZone.jsx** — **FileUploadZone** (React component) — Drag-and-drop PDF upload with size validation and error toasts.

C.4 Important Code Snippets

The next six subsections show small pieces of the actual code for the parts of the system that carry the most work. Each snippet has a short introduction that explains what the code does and why it was written this way. The snippets are kept short on purpose, we picked only the parts that show the main design decisions, not every helper function.

C.4.1 The Hand-off Contract (Pydantic Schema)

The “**Blueprint**” schema is the strict shape that Gemini has to produce. If Gemini returns something with a different shape (a wrong type, a missing field, a bad key name), Pydantic raises a “**ValidationError**”. The backend catches it and asks Gemini to try again. This is the piece that keeps the AI honest and stops bad data from moving into the code generation phase.

```
# app/models/blueprint.py
from pydantic import BaseModel, Field
from typing import List, Literal

class ModelField(BaseModel):
    name: str
    type: Literal["str", "int", "float", "bool", "datetime"]
    required: bool = True
    unique: bool = False

class DatabaseModel(BaseModel):
    name: str
    fields: List[ModelField]

class APIRoute(BaseModel):
    path: str
    method: Literal["GET", "POST", "PUT", "DELETE"]
    request_schema: str | None = None
    response_schema: str
    auth_required: bool = True

class UIComponent(BaseModel):
    name: str
    page: str
    props: List[str] = []

class Blueprint(BaseModel):
    project_name: str = Field(..., min_length=1)
    models: List[DatabaseModel]
    routes: List[APIRoute]
    components: List[UIComponent]
    seed_data: dict = {}
```

C.4.2 Gemini Clarification Agent

When the Validator finds missing fields in the requirements contract, this function picks the single most important question to ask the user. The answer is always one of the multiple-choice options, so the user does not have to type a long answer. This is the change we made after peer testing showed that open-ended questions were causing user fatigue.

```
# app/ai/gemini/agents/clarification.py
import json
from app.ai.gemini.client import gemini_client
from app.models.requirements import ClarificationQuestion

CLARIFICATION_PROMPT = """
You are given a JSON requirements contract and a list of missing fields.
Pick the SINGLE most important field to ask about, and produce a
multiple-choice question with 2 to 4 options.
Return ONLY valid JSON:
{"field": str, "question": str, "options": [str, ...]}
"""

def pick_next_question(contract: dict, missing_fields: list[str]) -> dict:
    payload = {"contract": contract, "missing_fields": missing_fields}
    response = gemini_client.generate(
        CLARIFICATION_PROMPT + "\n\n" + json.dumps(payload)
    )
    parsed = json.loads(response.strip())
    validated = ClarificationQuestion.model_validate(parsed)
    return validated.model_dump()
```

C.4.3 Debugger Strategy Ladder

This is the main function inside the Debugger. It always tries the cheapest fix first (Strategy 1, no LLM tokens spent). If that does not clear the errors, it tries a smaller LLM fix on just one file (Strategy 2). Only if that also fails does it try the expensive whole-project fix (Strategy 3). Between each step, the Tester runs again to check whether the fix worked. Before every fix we take a snapshot of the project, so if the fix makes things worse we can roll back.

```
# app/ai/claude/agents/debugger.py
def repair_project(project_dir: str, test_result: TestResult, cycle: int,
                  project_id: str) -> RepairResult:
    take_snapshot(project_dir, cycle)

    # Strategy 1: algorithmic fixers (no LLM tokens spent)
    if run_algorithmic_fixers(project_dir, test_result.errors):
        if rerun_tests(project_dir).passed:
            return RepairResult(success=True, strategy=1)

    # Strategy 2: targeted single-file LLM regeneration
    failing_file = parse_failing_file(test_result.trace)
    if failing_file:
        regenerate_single_file(project_dir, failing_file, test_result.errors)
        if rerun_tests(project_dir).passed:
            return RepairResult(success=True, strategy=2)

    # Strategy 3: whole-project agentic fix (capped at 5 per project)
    if agentic_uses[project_id] < 5:
        agentic_fix_project(project_dir, test_result)
        agentic_uses[project_id] += 1
        if rerun_tests(project_dir).passed:
            return RepairResult(success=True, strategy=3)

    # All three strategies failed - rollback to the last-good snapshot
    rollback_snapshot(project_dir, cycle)
    return RepairResult(success=False, strategy=None)
```

C.4.4 Subprocess Sandbox Runner

Each of the 10 test stages runs inside its own subprocess. Every subprocess has a wall-clock timeout of 45 seconds. If a stage takes longer than that, we kill the process. This way, a badly generated file (for example, one with an infinite loop) cannot lock up the main FastAPI server. Every project gets its own temporary directory so the generated code cannot touch anything outside its own workspace.

```
# app/services/sandbox_runner.py
import subprocess, tempfile, asyncio
from pathlib import Path

class SandboxRunner:
    TIMEOUT_SECONDS = 45

    async def run_test_stage(self, project_dir: Path, stage: str) -> StageResult:
        cmd = self.STAGE_COMMANDS[stage]
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            cwd=project_dir,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
        )
        try:
            stdout, stderr = await asyncio.wait_for(
                proc.communicate(), timeout=self.TIMEOUT_SECONDS
            )
            return StageResult(
                stage=stage,
                passed=(proc.returncode == 0),
                stdout=stdout.decode(errors="replace"),
                stderr=stderr.decode(errors="replace"),
            )
        except asyncio.TimeoutError:
            proc.kill()
            return StageResult(stage=stage, passed=False, stderr="TIMEOUT")

    def create_workspace(self) -> Path:
        return Path(tempfile.mkdtemp(prefix="demaestro_"))
```

C.4.5 Vercel Deployer

This function pushes the whole generated project to Vercel through its Deployments API. It also sets the Neon Postgres connection string as an environment variable, so the generated app can talk to its own isolated database. After the upload, we poll every 5 seconds until Vercel reports that the build is READY. If Vercel reports ERROR, or if we time out after 5 minutes, we raise an error and the frontend shows a failed status.

```
# app/services/vercel_deployer.py
import httpx, asyncio

async def deploy_project(project_id: str, files: list, neon_url: str) -> str:
    slug = slugify(project_id)
    vercel_files = [{"file": f.path, "data": f.content} for f in files]
    env_vars = {"DATABASE_URL": neon_url}

    resp = await httpx.AsyncClient().post(
        f"{VERCEL_API}/deployments",
        json={
            "name": slug,
            "files": vercel_files,
            "target": "production",
            "env": env_vars,
            "projectSettings": {"framework": "vite"},
        },
        headers={"Authorization": f"Bearer {VERCEL_TOKEN}"},
    )
    deployment = resp.json()

    # Poll every 5 seconds until READY (max 5 minutes)
    for _ in range(60):
        state = await get_deployment_state(deployment["id"])
        if state == "READY":
            return f"https://{deployment['url']}"
        if state == "ERROR":
            raise DeploymentError(deployment)
        await asyncio.sleep(5)
    raise DeploymentError("Timed out waiting for READY state")
```

C.4.6 Frontend Deployment Polling

The “**DeploymentPanel**” is the React component that shows the deployment status to the user. It polls the backend every 4 seconds while a project is deploying, and stops polling once the status is “**ready**”, “**ready_degraded**”, or “**failed**”. The “**pollNonce**” counter is used to force the polling loop to restart after a Redeploy click. Without this counter, the polling would stay stuck on the old “**failed**” state and never pick up the new deployment.

```
// frontend/src/components/deployment/DeploymentPanel.jsx
const [status, setStatus] = useState(null)
const [url, setUrl] = useState(null)
const [pollNonce, setPollNonce] = useState(0)

useEffect(() => {
  if (!projectId) return
  let cancelled = false
  let timer = null

  const tick = async () => {
    if (cancelled) return
    try {
      const r = await api.get(`/projects/${projectId}/deployment-status`)
      if (cancelled) return
      setStatus(r.data.status)
      setUrl(r.data.url)
      if (!['ready', 'ready_degraded', 'failed'].includes(r.data.status)) {
        timer = setTimeout(tick, 4000)
      }
    } catch {
      timer = setTimeout(tick, 8000)
    }
  }
  tick()
  return () => { cancelled = true; clearTimeout(timer) }
}, [projectId, pollNonce])

const handleRedeploy = async () => {
  setStatus('deploying')
  setUrl(null)
  setPollNonce((n) => n + 1) // force the polling effect to restart
  await api.post(`/projects/${projectId}/deploy`)
  setPollNonce((n) => n + 1) // pick up the first state transition immediately
}
```

